

Ruby Tooling Workshop for Junior Developers

Project: `palindrome_cli` – A command-line tool to check palindromes, view history, and get stats.

Detailed Exercise

1. Welcome & Why Tooling Matters

Talking Points:

- Professional Ruby development requires consistency across machines, reproducible setups, and automation.
 - The tools in this workshop form the foundation used in almost every real Ruby/Rails project.
-

2. rbenv – Ruby Version Management

Why rbenv?

rbenv lets you easily switch between different Ruby versions per project, preventing conflicts when working on multiple apps that require different Ruby versions. It keeps your system clean and ensures every developer on a team uses the exact same Ruby version.

Intro Guide:

- rbenv GitHub README – Key sections: “Why rbenv?” and “Installation” (explains version isolation benefits).

Hands-on Steps:

1. Install rbenv + ruby-build:

```
Bash

# macOS
brew install rbenv ruby-build
echo 'eval "$(rbenv init -)"' >> ~/.zshrc
```

2. Install Ruby and set project version:

```
Bash

mkdir palindrome_cli && cd palindrome_cli
echo "3.3.6" > .ruby-version
rbenv install --skip-existing
```

3. Verify:

```
Bash

ruby -v && rbenv versions
```

Challenge:

Create a second folder called `another_project` with a `.ruby-version` file specifying Ruby 3.2.6 (or another available version). Switch between the two projects and verify that each uses its own Ruby version using `ruby -v`.

3. Bundler – Dependency Management

Why Bundler?

Bundler manages your project's gems and ensures everyone on the team (or on CI servers) installs the exact same versions, avoiding the classic "it works on my machine" problem. It also handles gem dependencies intelligently and creates a `Gemfile.lock` for reproducibility.

Intro Guide:

- [Bundler.io: Getting Started](#) – Key sections: "Why Bundler?" and "The Gemfile" (highlights reproducibility and dependency resolution).

Hands-on Steps:

1. Initialize:

```
Bash

bundle init
```

2. Update Gemfile :

```
Ruby
source "https://rubygems.org"

ruby "~> 3.3.0"

gem "dry-cli", "~> 1.0"
gem "rspec", "~> 3.13"
gem "rake", "~> 13.2"
gem "rubocop", "~> 1.65", require: false
gem "pry", "~> 0.14", require: false
```

3. Install and create structure:

```
Bash

bundle install
mkdir -p lib spec bin
```

4. Create lib/palindrome_cli.rb:

```
Ruby

class PalindromeCli
  def palindrome?(string)
    return false if string.nil? || string.empty?

    cleaned = string.downcase.gsub(/[^a-z0-9]/, '')
    cleaned == cleaned.reverse
  end
end
```

Challenge:

Add the gem `tty-prompt (~> 0.23)` to your Gemfile (with `require: false`). Run `bundle install`, then use `bundle exec pry` to test that requiring it works. Observe how `Gemfile.lock` changes.

4. Rake – Task Automation

Why Rake?

Rake is Ruby's built-in task runner that lets you automate repetitive jobs (running tests, linting, starting servers, etc.) with simple, readable Ruby code instead of scattered shell scripts. It's the standard way Ruby projects define common development and deployment tasks.

Intro Guide:

- Rake Official Documentation – Key sections: “Introduction” and “Defining Tasks” (shows why Ruby-based automation is preferred over shell scripts).

Hands-on Steps:

Create Rakefile :

```
Ruby

require 'rspec/core/rake_task'
require 'rubocop/rake_task'

task default: %i[spec rubocop]

RSpec::Core::RakeTask.new(:spec)
RuboCop::RakeTask.new

desc "Run the CLI"
task :run do
  ruby "bin/palindrome"
end

desc "Open console"
task :console do
  exec "bundle exec pry -r ./lib/palindrome_cli"
end
```

Create placeholder `bin/palindrome` and make executable:

```
Bash
```

```
touch bin/palindrome
chmod +x bin/palindrome
```

Note on RuboCop

RuboCop is a static code analyzer and formatter for Ruby. It enforces consistent code style, catches common bugs, and helps keep your codebase clean and professional — exactly what teams expect in real Ruby projects.

You can run it directly with:

```
Bash
bundle exec rubocop
```

or via the Rake task we just set up:

```
Bash
bundle exec rake rubocop
```

Challenge:

Run RuboCop on your project (`bundle exec rubocop`). Fix any style violations it reports. Use `bundle exec rubocop --autocorrect` (or `--auto-correct`) to automatically fix what it can. Re-run until it passes cleanly.

5. RSpec – Testing

Why RSpec?

RSpec provides a readable, expressive way to write tests that act as both verification and living documentation for your code. It encourages behavior-driven development (BDD) and is the most widely used testing framework in the Ruby community.

Intro Guide:

- RSpec Documentation – Getting Started – Key sections: “Describe/It Syntax” and “Why RSpec?” (explains expressive testing and BDD style).

Hands-on Steps:

1. Initialize:

```
Bash  
  
bundle exec rspec --init
```

2. Update .rspec :

```
text  
  
--format documentation  
--color
```

3. Create spec/palindrome_cli_spec.rb :

```
Ruby  
  
require 'palindrome_cli'  
  
RSpec.describe PalindromeCli do  
  let(:cli) { PalindromeCli.new }  
  
  describe "#palindrome?" do  
    it "returns true for simple palindromes" do  
      expect(cli.palindrome?("racecar")).to be true  
      expect(cli.palindrome?("radar")).to be true  
    end  
  
    it "returns false for non-palindromes" do  
      expect(cli.palindrome?("hello")).to be false  
      expect(cli.palindrome?("ruby")).to be false  
    end  
  
    it "ignores case and non-alphanumeric characters" do  
      expect(cli.palindrome?("A man, a plan, a canal: Panama")).to be true  
      expect(cli.palindrome?("Was it a rat I saw?")).to be true  
    end  
  end  
end
```

1. Run:

```
Bash  
  
bundle exec rspec  
bundle exec rake
```

Challenge: Adding New Tests (Edge Cases)

Add a new test context for edge cases. Write tests that verify `palindrome?` returns `false` when given `nil` or an empty string. Run the tests — they should already pass with the existing implementation. This is great practice for extending test coverage.

Challenge (Test-First TDD with History):

Test-first (also called Red-Green-Refactor) means you write the test *before* you write the code that makes it pass. This ensures your tests are actually validating the behavior and forces you to think about the interface first.

Follow these steps in order:

1. Write new tests **first** for the history feature (they should fail – this is the “Red” phase):
 - Add tests that verify the class now maintains a history (an array of checks with input, result, and timestamp).
 - Add tests confirming that only valid inputs are added to history.
2. Then implement the history feature in `lib/palindrome_cli.rb` (add `@history` in `initialize` and push entries in `palindrome?` only for valid inputs) until all tests pass (“Green”).
3. (Optional) Clean up the code if needed (“Refactor”).

Run `bundle exec rspec` frequently to watch the tests go from red to green.

6. dry-cli + Bonus Tools

Why dry-cli?

`dry-cli` is a lightweight, fast, and modular command-line parsing library that encourages clean, maintainable CLI code without the heavy magic of Thor. It’s part of the excellent `dry-rb` ecosystem and scales well from simple scripts to complex tools.

Intro Guide:

- dry-cli Official Documentation – Key sections: “Getting Started” and “Commands” (highlights lightweight design and modularity).

Implementation:

Update bin/palindrome :

```
Ruby
```

```

#!/usr/bin/env ruby
require "bundler/setup"
require "dry/cli"
require_relative "../lib/palindrome_cli"

module Commands
  class Check < Dry::CLI::Command
    desc "Check if a string is a palindrome"

    argument :string, required: false, desc: "The string to check"

    def call(string: nil, **)
      cli = PalindromeCli.new

      if string
        result = cli.palindrome?(string)
        display_result(string, result, cli)
      else
        puts "No argument provided - running in basic mode (extend in challenge!)"
        # Basic single-run fallback for now
      end
    end

    private

    def display_result(string, result, cli)
      puts "\n✅ \"#{string}\" is#{result ? '' : ' NOT'} a palindrome!"

      if cli.history.any?
        puts "\nHistory (this session):"
        cli.history.each do |entry|
          status = entry[:is_palindrome] ? "✅" : "❌"
          puts "  #{status} #{entry[:input]}"
        end
      end
    end
  end
end

Dry::CLI.new(Commands).call

```

Test:

Bash

```
bundle exec bin/palindrome "A man, a plan, a canal: Panama"
```

Challenge:

Extend the CLI so that if no string is passed on the command line (`bin/palindrome`), it uses `TTY::Prompt` to enter an interactive loop allowing multiple palindrome checks until the user types “exit”. Also add a `history` sub-command that prints only the full history with timestamps. Register both commands properly in the `Dry::CLI.new` setup.